

JAVA PROGRAMMING MASTERCLASS

多執行緒

「讓程式同時做很多事：執行緒的建立、同步與通信」

[← 返回目錄](#)

章節大綱

第一部分：執行緒基礎

- 21-1 Program / Process / Thread
- 21-2 多工作業 Multitasking
- 21-3 Java 的多執行緒
- 21-4 執行緒的生命週期

第二部分：建立執行緒

- 21-5 建立執行緒（三種方法）
- 21-6 Java 執行緒工作原理

第三部分：執行緒控制

- 21-7 Thread.sleep()
- 21-8 Thread.join()
- 21-9 setPriority() 優先順序
- 21-10 setDaemon() 守護執行緒

第四部分：同步與進階

- 21-11 synchronized 同步
- 21-12 匿名類別
- 21-13 同步區塊
- 21-14 同步靜態方法
- 21-15 Deadlock 死結
- 21-16 wait() / notify()

第一部分

執行緒基礎

21-1 Program 、 Process 、 Thread

概念	說明	比喻
Program	儲存在磁碟上的靜態指令集合 (.class 檔案)	食譜書
Process	正在執行中的程式，擁有獨立的記憶體空間	正在做菜的廚房
Thread	行程內的最小執行單位，共享行程記憶體	廚房裡的廚師
比較項目	Process	Thread
記憶體	各自獨立	共享同一行程
建立成本	較高 (重量級)	較低 (輕量級)
通訊方式	IPC (較複雜)	直接存取共享變數
隔離性	完整隔離	需要同步保護

21-2 多工作業 Multitasking

兩種多工類型：

類型	說明	範例
Process-based	同時執行多個獨立程式	瀏覽器 + 文書軟體 + 音樂播放器
Thread-based	同一程式內同時執行多個任務	下載檔案同時更新進度條

為何需要多執行緒？

💡 核心優勢：充分利用 CPU 多核心，提升應用程式回應性與吞吐量

典型應用場景：

- 網頁伺服器同時處理多個請求
- GUI 應用背景下載不阻塞畫面
- 資料庫並行查詢

21-3 Java 的多執行緒

```
1 public class ThreadInfo {
2     public static void main(String[] args) {
3         Thread t = Thread.currentThread();
4         System.out.println("名稱: " + t.getName());
5         System.out.println("優先: " + t.getPriority());
6         System.out.println("狀態: " + t.getState());
7     }
8 }
9 // 輸出: 名稱: main / 優先: 5 / 狀態: RUNNABLE
```

相關 API

說明

`java.lang.Thread`

執行緒類別

`java.lang.Runnable`

任務介面 (`@FunctionalInterface`)

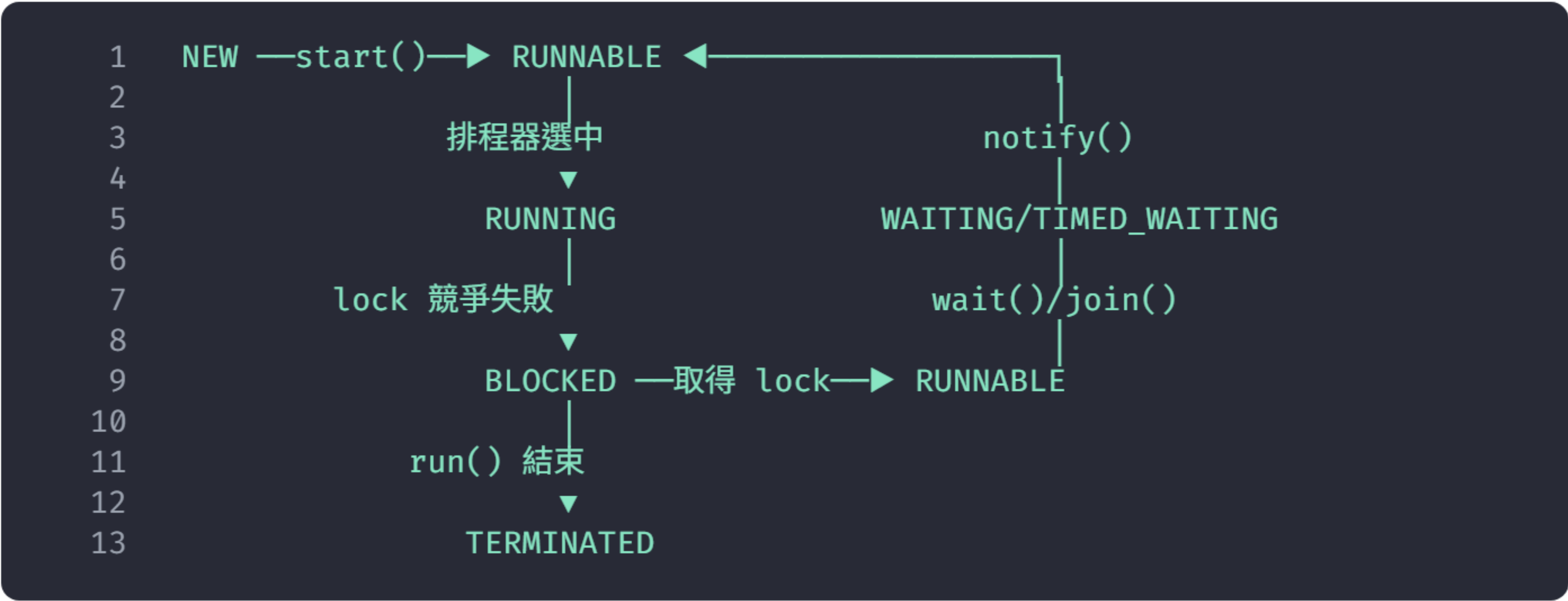
`java.lang.Object`

`wait()` / `notify()` 方法

`java.util.concurrent.*`

進階並發工具 (JDK 5+)

21-4 執行緒的生命週期



狀態	說明	觸發方式
NEW	已建立但未啟動	new Thread()
RUNNABLE	可執行或執行中	start()
BLOCKED	等待取得 monitor lock	競爭 synchronized

第二部分

建立執行緒

21-5 方法一：繼承 Thread 類別

```
1  class MyThread extends Thread {  
2      private String taskName;  
3  
4      public MyThread(String name) { this.taskName = name; }  
5  
6      @Override  
7      public void run() {  
8          for (int i = 1; i ≤ 3; i++)  
9              System.out.println(taskName + " - 步驟 " + i);  
10     }  
11 }
```

```
1  public class Main {  
2      public static void main(String[] args) {  
3          MyThread t1 = new MyThread("執行緒A");  
4          MyThread t2 = new MyThread("執行緒B");  
5          t1.start();    // ← 呼叫 start(), 不是 run()!  
6          t2.start();  
7      }  
8  }
```

21-5 方法二：實作 Runnable 介面

```
1  class PrintTask implements Runnable {
2      private String message;
3
4      public PrintTask(String msg) { this.message = msg; }
5
6      @Override
7      public void run() {
8          for (int i = 1; i ≤ 3; i++)
9              System.out.println(message + " #" + i);
10     }
11 }
```

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable task = new PrintTask("Hello");
4          Thread t = new Thread(task);
5          t.start();
6      }
7  }
```


21-5 方法三：Lambda 表達式 (Java 8+)

最簡範例：

```
1  Thread t = new Thread(() → System.out.println("Lambda 執行緒！"));
2  t.start();
```

完整範例：

```
1  Thread t1 = new Thread(() → {
2      for (int i = 1; i ≤ 3; i++)
3          System.out.println("任務1 - " + i);
4  }, "Worker-1");
5
6  Thread t2 = new Thread(() → {
7      for (int i = 1; i ≤ 3; i++)
8          System.out.println("任務2 - " + i);
9  }, "Worker-2");
10
11 t1.start();
12 t2.start();
```

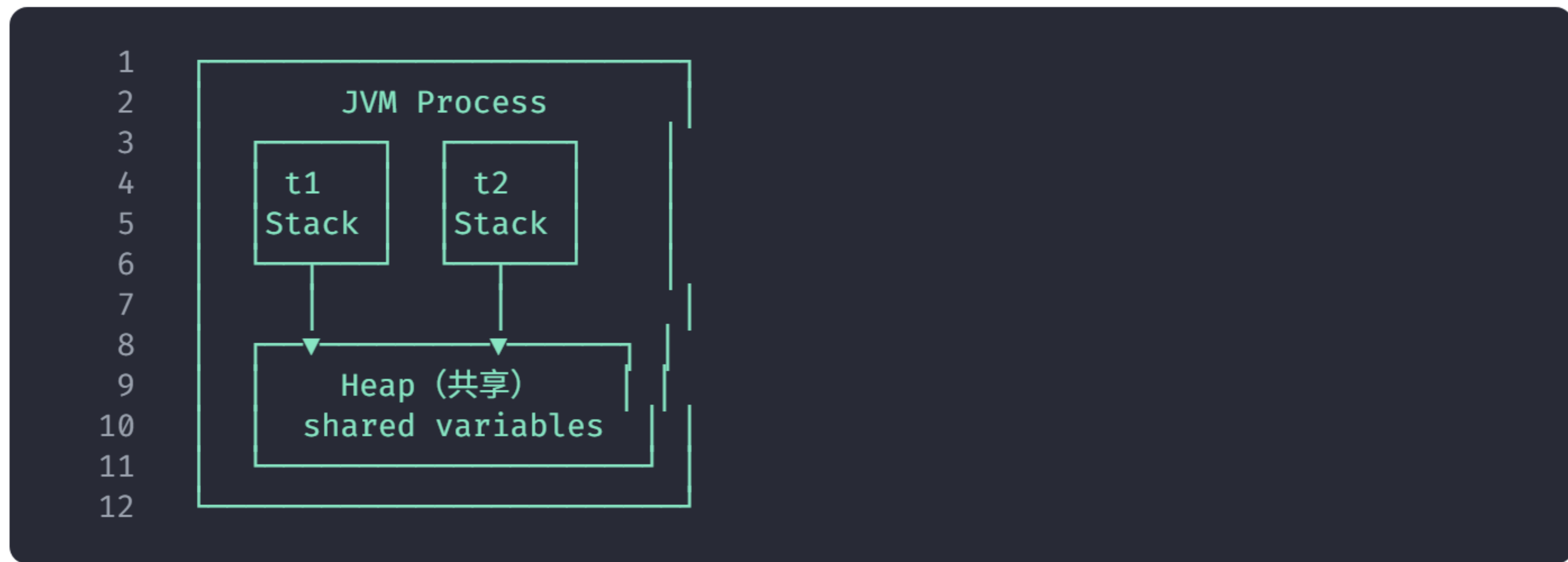
21-5 三種方法比較

方法	繼承 Thread	實作 Runnable	Lambda
Java 版本	全版本	全版本	Java 8+
繼承限制	無法再繼承	無限制	無限制
程式碼量	多	中	少
任務/執行緒分離	否	是	是
適用場景	需覆寫 Thread 方法時	一般推薦	簡單任務

💡 最佳實踐：優先使用 Lambda 或 Runnable；只有需要覆寫 **Thread** 的其他方法時才繼承

21-6 Java 執行緒的工作原理

每個執行緒有獨立的 Stack，但共享 Heap 中的物件：



方法

說明

`getName()` / `getId()`

取得名稱 / ID

`getState()`

取得目前狀態

第三部分

執行緒控制

21-7 Thread.sleep() – 讓執行緒睡眠

最簡範例：

```
1 Thread.sleep(1000); // 暫停 1 秒 (1000 毫秒)
```

完整倒數範例：

```
1 public class SleepDemo {  
2     public static void main(String[] args) throws InterruptedException  
{  
3         for (int i = 5; i ≥ 1; i--) {  
4             System.out.println("倒數：" + i);  
5             Thread.sleep(1000);  
6         }  
7         System.out.println("發射！");  
8     }  
9 }
```

方法簽章

說明

`sleep(long millis)`

暫停指定毫秒

21-8 Thread.join() – 等待執行緒完成

最簡範例：

```
1  t1.join(); // 等待 t1 結束後再繼續
```

完整範例：

```
1  public class JoinDemo {
2      public static void main(String[] args) throws InterruptedException
3      {
4          Thread worker = new Thread(() → {
5              System.out.println("Worker 開始處理 ... ");
6              try { Thread.sleep(2000); } catch (InterruptedException e)
7              {}
8              System.out.println("Worker 完成!");
9          });
10     worker.start();
11     worker.join(); // main thread 等待 worker 結束
12     System.out.println("主程式繼續執行");
13 }
```


21-9 setPriority() – 執行緒優先順序

```
1 Thread t = new Thread(() → System.out.println("執行中"));
2 t.setPriority(Thread.MAX_PRIORITY); // 設為最高優先
3 t.start();
```

常數	值	說明
Thread.MIN_PRIORITY	1	最低優先權
Thread.NORM_PRIORITY	5	預設優先權
Thread.MAX_PRIORITY	10	最高優先權
方法	說明	
setPriority(int p)	設定優先權 (1-10)	
getPriority()	取得目前優先權	

21-10 setDaemon() – 守護執行緒

最簡範例：

```
1  Thread t = new Thread(() → { /* 背景任務 */ });
2  t.setDaemon(true); // 必須在 start() 前設定
3  t.start();
```

完整範例：

```
1  Thread daemon = new Thread(() → {
2      while (true) {
3          System.out.println("Daemon 監控中 ... ");
4          try { Thread.sleep(500); } catch (InterruptedException e) { br
eak; }
5      }
6  });
7  daemon.setDaemon(true);
8  daemon.start();
9  Thread.sleep(1500);
10 System.out.println("Main 結束，Daemon 自動停止");
```


第四部分

同步機制

21-11 為什麼需要同步？

不安全的計數器 (`count++` 是三個步驟：讀取 → 加 1 → 寫回)：

```
1  class Counter {  
2      private int count = 0;  
3      public void increment() { count++; } // 非原子操作！  
4  }
```

```
1  Thread-1: 讀取 count=0  
2  Thread-2: 讀取 count=0  
3  Thread-1: 寫入 count=1  
4  Thread-2: 寫入 count=1 ← 期望是 2，結果卻是 1！
```

● **Race Condition (競爭條件)**：多個執行緒同時存取並修改共享資料，執行結果依賴執行順序，導致不可預測的錯誤。

21-11 synchronized 關鍵字

修正後的安全計數器：

```
1  class SafeCounter {
2      private int count = 0;
3
4      public synchronized void increment() {
5          count++; // 同一時刻只有一個執行緒能進入
6      }
7
8      public synchronized int getCount() { return count; }
9  }
```

```
1  SafeCounter counter = new SafeCounter();
2  Thread t1 = new Thread(() → { for(int i=0; i<1000; i++) counter.increment(); });
3  Thread t2 = new Thread(() → { for(int i=0; i<1000; i++) counter.increment(); });
4  t1.start(); t2.start();
5  t1.join(); t2.join();
6  System.out.println("結果: " + counter.getCount()); // 必定是 2000
```


21-12 匿名類別 Anonymous Class

不需要另外定義類別，直接在使用處定義並建立物件：

```
1 // 用匿名類別實作 Runnable
2 Thread t = new Thread(new Runnable() {
3     @Override
4     public void run() {
5         System.out.println("匿名類別執行緒：" +
6             Thread.currentThread().getName());
7     }
8 });
9 t.start();
```

```
1 // Lambda 等效寫法 (Java 8+)
2 Thread t2 = new Thread(
3     () → System.out.println("Lambda 執行緒")
4 );
5 t2.start();
```

💡 比較：匿名類別 vs Lambda — 兩者效果相同，但 Lambda 語法更簡潔。Java 8 後推薦使用 Lambda。

21-13 同步區塊 Synchronized Block

比同步方法更精細，只鎖定關鍵區域：

```
1  class PartialSync {
2      private int shared = 0;
3      private final Object lock = new Object();
4
5      public void process() {
6          // 非同步的準備工作（可並行執行）
7          System.out.println("準備中 ... ");
8
9          synchronized (lock) {    // 只對關鍵部分加鎖
10             shared++;
11             System.out.println("共享值：" + shared);
12         }
13
14         System.out.println("收尾中 ... ");
15     }
16 }
```

比較

synchronized 方法

synchronized 區塊

鎖定範圍

整個方法

指定區塊

21-14 同步靜態方法

靜態方法的 `synchronized` 使用 **Class** 層級的鎖 (不是 `this`) :

```
1  class ClassLevelSync {
2      private static int instanceCount = 0;
3
4      // 鎖定的是 ClassLevelSync.class, 所有實例共用同一把鎖
5      public static synchronized void register() {
6          instanceCount++;
7      }
8
9      public static synchronized int getCount() {
10         return instanceCount;
11     }
12 }
```

比較	實例方法 <code>synchronized</code>	靜態方法 <code>synchronized</code>
鎖	this (物件本身)	ClassName.class (Class 物件)
不同物件	不同鎖，可並行	全局同一把鎖

第五部分

進階議題

21-15 死結 Deadlock – 成因

死結發生的四個必要條件（全部同時成立才發生）：

條件	說明
互斥	資源同時只能被一個執行緒持有
持有並等待	執行緒持有資源同時等待其他資源
不可搶佔	資源只能由持有者主動釋放
循環等待	執行緒形成循環等待鏈

- 1 Thread-1 持有 lockA，等待 lockB
- 2 Thread-2 持有 lockB，等待 lockA
- 3 ↕ 互相等待，永遠不會結束

● **Deadlock**：所有涉及的執行緒都無法繼續執行，程式進入永久停止狀態。

21-15 Deadlock – 程式碼示範

```
1  class DeadlockDemo {
2      static Object lockA = new Object();
3      static Object lockB = new Object();
4
5      public static void main(String[] args) {
6          Thread t1 = new Thread(() → {
7              synchronized (lockA) {
8                  System.out.println("T1 取得 lockA");
9                  try { Thread.sleep(100); } catch (InterruptedException
e) {}
10                 synchronized (lockB) { System.out.println("T1 取得 lock
B"); }
11             }
12         });
13         Thread t2 = new Thread(() → {
14             synchronized (lockB) {                // 順序相反！
15                 System.out.println("T2 取得 lockB");
16                 synchronized (lockA) { System.out.println("T2 取得 lock
A"); }
17             }
18         });
19         t1.start(); t2.start();
20     }
21 }
```

21-16 wait() / notify() / notifyAll()

這三個方法定義在 **Object** 類別，必須在 **synchronized** 區塊內使用：

方法	說明
wait()	釋放 lock，進入 WAITING 狀態等待通知
wait(long ms)	釋放 lock，最多等待指定毫秒
notify()	喚醒一個正在 wait 的執行緒（隨機選擇）
notifyAll()	喚醒所有正在 wait 的執行緒

💡 與 **sleep()** 的差異：

sleep()：不釋放 lock，時間到自動恢復

wait()：釋放 lock，需要別人 **notify()** 才能恢復

21-16 生產者-消費者模式

```
1  class SharedBuffer {
2      private int data = -1;
3      private boolean hasData = false;
4
5      public synchronized void produce(int val) throws InterruptedException {
6          while (hasData) wait();          // 已有資料，等消費
7          data = val;
8          hasData = true;
9          System.out.println("生產: " + val);
10         notifyAll();                     // 通知消費者
11     }
12
13     public synchronized int consume() throws InterruptedException {
14         while (!hasData) wait();          // 沒有資料，等生產
15         hasData = false;
16         System.out.println("消費: " + data);
17         notifyAll();                     // 通知生產者
18         return data;
19     }
20 }
```


練習一：計時器執行緒

任務說明

設計一個多執行緒計時器應用程式：

1. 建立一個 `CountdownTask` 類別，實作 `Runnable` 介面
2. 建構子接受整數 `seconds` (倒數秒數) 和 `String name` (計時器名稱)
3. `run()` 方法每隔 1 秒印出 `"[name] 剩餘 x 秒"`，結束時印出 `"[name] 計時完成！"`
4. 在 `main` 中同時啟動 3 個不同秒數的計時器 (3 秒、5 秒、7 秒)
5. 使用 `join()` 等待所有計時器完成後印出 `"所有計時器完成"`

練習一：解題提示

1. Runnable 骨架：

```
class CountdownTask implements Runnable {  
    private int seconds;  
    private String name;  
    public void run() { /* 迴圈 + sleep(1000) */ }  
}
```

2. sleep() 的例外處理：在 `run()` 內用 try-catch 包住 `Thread.sleep(1000)`

3. 建立並啟動：

```
Thread t1 = new Thread(new CountdownTask(3, "Timer-3"));  
t1.start();
```

4. join() 的位置：所有 `start()` 呼叫完後，再依序 `t1.join(); t2.join(); t3.join();`

5. 進階挑戰：改用 Lambda 實作同樣功能

練習二：安全的銀行帳戶

任務說明

模擬多執行緒存取提款，練習同步機制：

1. 建立 `BankAccount` 類別，包含 `balance` (餘額) 欄位
2. 實作 `synchronized` 的 `deposit(int amount)` 和 `withdraw(int amount)` 方法
 - `withdraw` 若餘額不足，印出警告並返回 `false`
3. 初始餘額設為 1000
4. 建立 5 個「存款執行緒」(各存 200) 和 5 個「提款執行緒」(各提 300)
5. 所有執行緒完成後，印出最終餘額

驗證：先去掉 `synchronized` 執行幾次，觀察不穩定的結果，再加回去驗證一致性。

練習二：解題提示

1. BankAccount 骨架：

```
class BankAccount {  
    private int balance;  
    public synchronized void deposit(int amount) { balance += amount; }  
    public synchronized boolean withdraw(int amount) {  
        if (balance < amount) { System.out.println("餘額不足"); return false; }  
        balance -= amount;  
        return true;  
    }  
    public synchronized int getBalance() { return balance; }  
}
```

2. 建立多個執行緒：

```
Thread[] threads = new Thread[10];  
for (int i = 0; i < 5; i++) {  
    threads[i] = new Thread(() → account.deposit(200));  
    threads[i+5] = new Thread(() → account.withdraw(300));  
}
```


練習三：生產者消費者佇列

任務說明

實作一個有界緩衝區 (Bounded Buffer) 的生產者消費者模式：

1. 建立 `BoundedQueue` 類別，內部使用 `LinkedList<Integer>`，最大容量為 5
2. 實作 `put(int item)`：若佇列滿，呼叫 `wait()` 等待；否則加入並呼叫 `notifyAll()`
3. 實作 `take()`：若佇列空，呼叫 `wait()` 等待；否則取出並呼叫 `notifyAll()`
4. 建立 2 個生產者 (各生產 10 個數字) 和 3 個消費者
5. 生產者完成後傳送結束訊號 (-1) 給消費者

重點觀察： 佇列永遠不會超過 5 個元素；程式能正常結束 (不會永遠 hang 住)

練習三：解題提示

```
1  class BoundedQueue {
2      private LinkedList<Integer> queue = new LinkedList<>();
3      private int capacity;
4      public BoundedQueue(int cap) { this.capacity = cap; }
5
6      public synchronized void put(int item) throws InterruptedException
7      {
8          while (queue.size() == capacity) wait();
9          queue.addLast(item);
10         notifyAll();
11     }
12
13     public synchronized int take() throws InterruptedException {
14         while (queue.isEmpty()) wait();
15         int val = queue.removeFirst();
16         notifyAll();
17         return val;
18     }
19 }
```

重點：

Q & A

課程結束

多執行緒：建立、控制、同步、通信